

AI-BASED PREDICTIVE TRANSFORMER HEALTH MONITORING AND PREVENTIVE PROTECTION SYSTEM USING ARDUINO UNO

A Project Report

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING
in
ELECTRICAL AND ELECTRONICS ENGINEERING

Submitted by

KARTHIKEYAN M

Register Number: 723723105010

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

V.S.B COLLEGE OF ENGINEERING TECHNICAL CAMPUS

COIMBATORE

ANNA UNIVERSITY, CHENNAI – 600 025

ACADEMIC YEAR: 2025 – 2026

III YEAR – VI SEMESTER

BONAFIDE CERTIFICATE

This is to certify that the Project report titled “AI-BASED PREDICTIVE TRANSFORMER HEALTH MONITORING AND PREVENTIVE PROTECTION SYSTEM USING ARDUINO UNO” is a bonafide record of work done by KARTHIKEYAN M (Register Number: 723723105010) of the Department of Electrical and Electronics Engineering, in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Electrical and Electronics Engineering of Anna University, Chennai, during the academic year 2025–2026 (III Year – VI Semester).

This project work has been carried out under my supervision and has not been submitted, either in part or in full, for the award of any other degree or diploma of this or any other institution.

Signature of Faculty Guide

Signature of Head of the Department

Submitted for the Viva-Voce examination held on _____.

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

I sincerely thank the management of V.S.B College of Engineering Technical Campus, Coimbatore, for providing the infrastructure and resources needed to carry out this project. I am grateful to my faculty guide for the continuous guidance, technical suggestions, and encouragement given throughout the design and testing of this system. Their inputs were especially helpful while working out the health score and risk prediction logic used in this project. I also thank the Head of the Department of Electrical and Electronics Engineering for the support and for making the department's resources available for this work. I extend my thanks to the teaching and non-teaching staff of the department for their assistance during the course of this project. Finally, I thank my parents and friends for their constant encouragement and support, without which this project would not have been completed successfully.

KARTHIKEYAN M

ABSTRACT

Transformer failure is one of the most disruptive and costly events in an electrical power system, and in most low-cost installations, protection is limited to a single-parameter thermal cut-off that reacts only after the temperature has already reached a dangerous level. This project presents the design and simulation of an AI-Based Predictive Transformer Health Monitoring and Preventive Protection System built around the Arduino UNO microcontroller.

The system uses a DHT22 Temperature Sensor to continuously monitor the transformer's operating temperature, and a potentiometer to simulate the variable electrical load on the transformer. The Arduino UNO reads both values in real time and uses them to calculate a Health Score, applying a penalty once the temperature exceeds 40°C and once the load exceeds 75%.

Based on the calculated health score, the system classifies the transformer's condition into one of four risk levels, LOW, MEDIUM, HIGH, or CRITICAL, and further derives a Failure Probability percentage and a Remaining Life estimate scaled from a 15-year baseline. All of these values, along with the fault count and relay status, are displayed on a rotating 16×2 I2C LCD dashboard.

Whenever the temperature crosses 45°C, a servo motor representing a cooling fan is activated with an angle proportional to the severity of the temperature rise. If the risk level reaches CRITICAL, a relay module automatically disconnects the simulated transformer supply, while an LED and buzzer provide an immediate visual and audible alert. A fault counter keeps a running record of how many times the transformer has entered a critical state.

Unlike conventional threshold-only protection systems, the proposed system gives a graded, predictive view of transformer health rather than a single pass-or-fail alarm. The complete hardware was designed and simulated using the Wokwi online simulator, and the program was developed in Embedded C using the Arduino IDE. All four risk conditions, along with the cooling, protection, and fault-counting behaviour, were tested and verified successfully.

The results confirm that the developed system is low-cost, easy to build, and effective at giving early warning of transformer stress well before a fault actually occurs, making it suitable for small distribution transformers, industrial units, and academic demonstrations of predictive maintenance concepts.

Keywords: Arduino UNO, DHT22 Sensor, Transformer Health Monitoring, Predictive Protection, Health Score, Risk Prediction, Relay Protection, Embedded System.

The developed system demonstrates that predictive monitoring techniques can significantly improve transformer reliability while reducing maintenance cost and unexpected failures.

TABLE OF CONTENTS

INTRODUCTION	10
1.1 Introduction.....	10
1.2 Problem Statement	10
1.3 Objectives	11
1.4 Scope of the Project.....	11
1.5 Applications	12
LITERATURE SURVEY	13
2.1 Introduction.....	13
2.2 Existing System	13
2.3 Limitations of Existing Systems.....	13
2.4 Proposed System	14
2.5 Comparison Table	14
HARDWARE DESCRIPTION	16
3.1 Arduino UNO	16
3.2 DHT22 Sensor	16
3.3 Potentiometer	17
3.4 LCD Display	17
3.5 Servo Motor	17
3.6 Relay Module	18
3.7 LED	18
3.8 Buzzer	18
3.9 Power Supply.....	18
3.10 Pin Configuration	18
SOFTWARE DESCRIPTION	20
4.1 Arduino IDE	20
4.2 Wokwi Simulator	20
4.3 Embedded C.....	20
4.4 Circuit Explanation	21
4.5 Working Principle	22
4.6 Flowchart	23
4.7 Algorithm.....	24
4.8 Program Description	25
SIMULATION AND IMPLEMENTATION	26
5.1 Simulation Setup	26
5.2 Circuit Diagram Explanation.....	27
5.3 Experimental Results	27
5.4 Testing Table	29
5.5 Discussion	29
ADVANTAGES, LIMITATIONS AND FUTURE SCOPE.....	31
6.1 Advantages.....	31
6.2 Limitations.....	31
6.3 Future Scope	31
CONCLUSION	33
REFERENCES	34

APPENDIX A.....	35
APPENDIX B.....	41

(Right-click the table above in Word and choose “Update Field” so page numbers reflect the final layout.)

LIST OF FIGURES

Reference No.	Description
Figure 4.1	Block Diagram of the Proposed System
Figure 4.2	Flowchart of System Operation
Figure 5.1	Complete Circuit Diagram – Wokwi Simulation
Figure 5.2	LCD Output – Normal (LOW Risk) Condition
Figure 5.3	LCD Output – Rising Temperature (MEDIUM/HIGH Risk)
Figure 5.4	LCD Output – Critical Condition (Relay Trip)

LIST OF TABLES

Reference No.	Description
Table 2.1	Comparison of Existing and Proposed System
Table 3.1	Arduino UNO Specifications
Table 3.2	DHT22 Sensor Specifications
Table 3.3	Potentiometer Specifications
Table 3.4	Pin Configuration
Table 4.1	Threshold Configuration
Table 5.1	Result Summary of Test Cases

CHAPTER 1

INTRODUCTION

1.1 Introduction

Transformers form one of the most critical links in any electrical power system, stepping voltage up or down so that power can be transmitted efficiently and delivered safely to homes and industries. Because transformers typically run continuously for years, they are exposed to constant thermal and electrical stress [4], and this stress accumulates gradually rather than causing a sudden, visible fault.

Most low-cost or small-substation protection systems only react once a single parameter, usually temperature, crosses one fixed limit. This kind of protection prevents an immediate disaster but gives no indication of how the transformer's condition is changing over time, and no early warning before the critical point is reached.

This project presents an AI-Based Predictive Transformer Health Monitoring and Preventive Protection System developed using the Arduino UNO microcontroller. The system continuously monitors temperature using a DHT22 sensor and load using a potentiometer, and combines both readings into a single Health Score rather than checking either parameter in isolation.

Based on this health score, the system classifies the transformer into one of four risk levels, LOW, MEDIUM, HIGH, or CRITICAL, and further estimates a Failure Probability and a Remaining Life, giving the operator a graded, predictive picture rather than a single pass-or-fail alarm. Whenever the condition becomes dangerous, the system automatically activates cooling and, if necessary, disconnects the supply using a relay, while alerting the operator through an LED, a buzzer, and a rotating LCD dashboard.

The system is low-cost, simple to build, and well suited both as an academic demonstration of predictive maintenance concepts and as a starting point for a real small-substation protection system.

1.2 Problem Statement

Unplanned transformer failure is costly and disruptive, and most low-cost protection systems used in small substations rely only on a single fixed temperature threshold. Once this threshold is crossed, the supply is cut off, but there is no indication of how close the transformer was to that point beforehand, and no separate handling for a transformer under high load, high temperature, or both at once.

There is also no concept of gradual health tracking in most of these systems. A transformer is treated as either healthy or faulty, with no intermediate warning stage, and no running record of how many times it has been pushed into a stressful condition.

This project addresses this gap by developing a low-cost Arduino-based system that combines temperature and load readings into a single health score, classifies the risk level in graded stages, estimates failure probability and remaining life, and takes automatic preventive action before the transformer reaches a truly critical condition.

1.3 Objectives

The main objectives of this project are as follows:

- To continuously monitor the transformer's operating temperature using a DHT22 sensor and its load using a potentiometer.
- To calculate a combined Health Score based on both temperature and load readings.
- To classify the transformer's condition into LOW, MEDIUM, HIGH, or CRITICAL risk levels based on the health score.
- To estimate the Failure Probability and the Remaining Life of the transformer from the calculated health score.
- To automatically activate a cooling mechanism and a protective relay disconnection whenever the condition becomes dangerous.
- To provide clear visual and audible alerts using an LED and buzzer, and to display all key readings on a 16×2 I2C LCD dashboard.
- To develop a low-cost, reliable predictive protection system using Arduino UNO, fully verifiable through simulation.

1.4 Scope of the Project

The developed system is intended for predictive health monitoring and preventive protection of small to medium transformers. It continuously evaluates temperature and load, calculates a graded health score, and triggers cooling or disconnection whenever the transformer moves into a risky operating state.

The project can be applied in the following areas:

- Small and medium distribution substations
- Industrial units with dedicated transformers

- Educational laboratories for demonstrating predictive maintenance concepts
- Rural or remote installations with limited technician availability

Future improvements such as IoT connectivity, cloud-based health trend logging, and a mobile alert application can further extend the reach of the system, as discussed in Chapter 6.

1.5 Applications

The major features and application areas of the developed system are summarised below:

- Real-time transformer temperature and load monitoring
 - Combined health score calculation instead of single-parameter checking
 - Four-level graded risk classification (LOW, MEDIUM, HIGH, CRITICAL)
 - Failure probability and remaining life estimation
 - Automatic proportional cooling activation
 - Automatic relay-based protective disconnection
 - Visual and audible alerts through LED and buzzer
 - Fault counter for tracking repeated critical events
 - Rotating LCD dashboard for continuous status display
 - Low-cost, Arduino-based design suitable for student projects and small substations
- The proposed system provides an economical, reliable, and scalable solution suitable for educational demonstrations, laboratory research, and small-scale industrial transformer monitoring applications.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

Transformer protection and condition monitoring have been studied extensively, since transformer failure directly affects power availability and safety. A wide range of monitoring approaches exist, from simple thermal cut-offs to full SCADA-based substation monitoring, each with its own cost and complexity trade-offs.

This chapter reviews the common categories of existing transformer monitoring and protection systems, identifies their limitations, and explains how these limitations motivated the design of the proposed system.

2.2 Existing System

Simple thermal cut-off systems use a temperature sensor or bimetallic switch to disconnect supply once a fixed set point is crossed. This method is reliable in a basic sense but reacts only after the danger point is reached, with no early warning stage.

Oil level and pressure-based protection is common in larger oil-filled transformers, tripping supply when oil level drops or internal pressure rises beyond safe limits [5]. This approach does not apply well to small, dry-type, or simulated transformer setups, and does not account for electrical loading at all.

SCADA-based monitoring, used in larger substations, sends sensor data to a control room for human observation. This gives good visibility but depends on expensive communication infrastructure and continuous human interpretation, making it impractical for small distribution transformers.

More recent IoT-based student and hobbyist projects send temperature readings to a mobile app or cloud dashboard. This is a step forward for remote visibility, but in most simpler implementations, the underlying logic is still a fixed-threshold comparison, with no combined health scoring or graded risk classification.

2.3 Limitations of Existing Systems

The following limitations were identified from the review of existing systems:

- Most systems monitor temperature alone, ignoring the effect of load.
- Protection is typically a single ON/OFF trip with no intermediate warning stage.

- There is no combined health indicator that reflects the interaction between temperature and load.
- No estimate of failure probability or remaining life is provided.
- Larger SCADA or IoT-based systems increase cost and depend on continuous connectivity.
- Most systems keep no record of how many times a transformer has entered a stressed condition.

2.4 Proposed System

The proposed AI-Based Predictive Transformer Health Monitoring and Preventive Protection System overcomes these limitations by combining temperature and load readings into a single Health Score, rather than checking either parameter alone.

Based on this health score, the system classifies the transformer into four risk levels, LOW, MEDIUM, HIGH, or CRITICAL, and further derives a Failure Probability percentage and a Remaining Life estimate, giving a graded and predictive picture of transformer condition rather than a single pass-or-fail alarm.

Whenever the temperature crosses a defined limit, a servo-driven cooling mechanism is activated automatically, and if the risk level reaches CRITICAL, a relay disconnects the supply while an LED and buzzer alert the operator. A fault counter tracks how many times the transformer has entered a critical state, giving a simple historical record for maintenance planning.

2.5 Comparison Table

Table 2.1 compares the proposed system with the existing approaches described above.

Feature	Existing Systems	Proposed System
Temperature Monitoring	Yes	Yes
Load Monitoring	No	Yes
Combined Health Score	No	Yes
Graded Risk Classification	No	Yes (LOW/MEDIUM/HIGH/CRITICAL)

Feature	Existing Systems	Proposed System
Failure Probability Estimate	No	Yes
Remaining Life Estimate	No	Yes
Automatic Cooling	Rare	Yes (Proportional)
Automatic Protection	Yes (Single Threshold)	Yes (Graded, CRITICAL-triggered)
Fault History Tracking	No	Yes (Fault Counter)
Arduino Based	Some Systems	Yes
Low Cost	Moderate	Yes

Table 2.1 Comparison of Existing and Proposed System

From the comparison above, the proposed system offers a more graded and predictive monitoring solution than the majority of existing low-cost transformer protection systems, while remaining simple and economical to build.

The review of existing literature clearly indicates the need for an intelligent low-cost predictive monitoring system capable of analysing multiple operating parameters simultaneously.

CHAPTER 3

HARDWARE DESCRIPTION

This chapter describes the hardware components used in the development of the transformer health monitoring system. Each component is explained along with its role in the overall system and its important technical specifications.

3.1 Arduino UNO

The Arduino UNO is the main controller used in this project. It is based on the ATmega328P microcontroller [1] and provides sufficient digital and analog input/output pins to interface with the sensor, potentiometer, and all output devices. It reads the temperature and load values, computes the health score and risk level, and drives the servo, relay, LED, buzzer, and LCD accordingly.

Parameter	Specification
Microcontroller	ATmega328P
Operating Voltage	5 V
Digital I/O Pins	14
Analog Input Pins	6
Flash Memory	32 KB
SRAM	2 KB
Clock Speed	16 MHz

Table 3.1 Arduino UNO Specifications

3.2 DHT22 Sensor

The DHT22 sensor is used to measure the temperature representing the transformer's operating condition. Compared to the simpler DHT11, the DHT22 offers a wider measurement range and better accuracy, and it sends calibrated digital data directly to the Arduino, avoiding the need for additional signal conditioning circuitry [2]. In the program, a reading is discarded if it comes back invalid, to avoid a single faulty reading being mistaken for a genuine temperature change.

Parameter	Specification
Operating Voltage	3.3 V – 6 V
Temperature Range	-40°C to 80°C
Accuracy	±0.5°C
Resolution	0.1°C
Response Time	~2 seconds

Table 3.2 DHT22 Sensor Specifications

3.3 Potentiometer

The potentiometer is used to simulate the variable electrical load on the transformer, since generating an actual variable high-current load safely within a simulator is not practical. Rotating the potentiometer changes the analog voltage read by the Arduino, and this value is mapped in software to a load percentage between 0% and 100%.

Parameter	Specification
Type	Linear Rotary Potentiometer
Resistance	10 kΩ
Operating Voltage	0 V – 5 V
Output	Analog (0–1023 via ADC)
Function	Load Simulation

Table 3.3 Potentiometer Specifications

3.4 LCD Display

A 16×2 I2C LCD display is used to present system information to the operator. It rotates through four screens, showing health score with risk level, temperature with load, remaining life with failure probability, and fault count with relay status. The I2C interface reduces wiring to just two communication lines, SDA and SCL, instead of the many parallel data lines needed by a standard LCD [3].

3.5 Servo Motor

A servo motor represents the transformer's cooling fan. Once the temperature crosses 45°C, the Arduino rotates the servo to an angle proportionally mapped between 90 and 180 degrees, so that the simulated fan runs harder as the temperature rises further, rather than simply switching fully on or off.

3.6 Relay Module

The relay module represents the main protective element of the system. It is kept in its energised, connected state during normal operation, and is de-energised by the Arduino the moment the risk level reaches CRITICAL, disconnecting the simulated transformer supply to prevent further stress.

3.7 LED

The LED provides a visual indication of a critical condition. It remains off during LOW, MEDIUM, and HIGH risk conditions, and switches on the moment the risk level reaches CRITICAL, at the same time the relay trips.

3.8 Buzzer

The buzzer provides an audible alert alongside the LED. It is switched on at the same instant as the LED, whenever the risk level reaches CRITICAL, so that the operator is alerted even without looking at the display.

3.9 Power Supply

The complete circuit is powered through the Arduino UNO's USB connection, which supplies the regulated 5V required by the microcontroller, the LCD, the servo, the relay module, the DHT22 sensor, and the potentiometer. A stable supply is important here, since voltage fluctuation can directly affect both the analog load reading and the timing-sensitive DHT22 communication. All ground connections are tied to a common rail to avoid floating ground potential differences that could introduce noise into the sensor readings.

3.10 Pin Configuration

Component	Arduino Pin
DHT22 Data Pin	D2
Potentiometer Wiper (SIG)	A0
LCD SDA	A4
LCD SCL	A5
Servo Signal (PWM)	D10
Relay Control (IN)	D11
LED	D8
Buzzer	D9

Table 3.4 Pin Configuration

CHAPTER 4

SOFTWARE DESCRIPTION

4.1 Arduino IDE

The Arduino IDE was used to write, compile, and upload the program to the Arduino UNO. It provides a simple text editor along with a compiler, uploader, and a large collection of open-source libraries, which simplified communication with the DHT22 sensor, the I2C LCD, and the servo motor in this project [9].

4.2 Wokwi Simulator

Wokwi is a free, browser-based simulator that was used to design, wire, and test the complete circuit for this project without needing physical hardware [10]. All components used in this project, the DHT22, potentiometer, I2C LCD, servo motor, relay module, LED, and buzzer, were connected virtually inside Wokwi, and the same Arduino code was used directly to test the system's behaviour under different simulated temperature and load conditions.

4.3 Embedded C

The program for this project was written in Embedded C, the standard language supported by the Arduino IDE. The code is structured into a ``setup()`` function, a one-time ``startupAnimation()`` sequence, and a ``loop()`` function that checks two independent timers, one for sensor evaluation every second and one for LCD refresh every three seconds, rather than relying on blocking ``delay()`` calls.

4.4 Circuit Explanation

The Arduino UNO acts as the central controller of the system. It receives temperature data from the DHT22 sensor and load data from the potentiometer, computes the health score and risk level, and drives the LCD, servo, relay, LED, and buzzer accordingly. The overall architecture is illustrated in Figure 4.1.

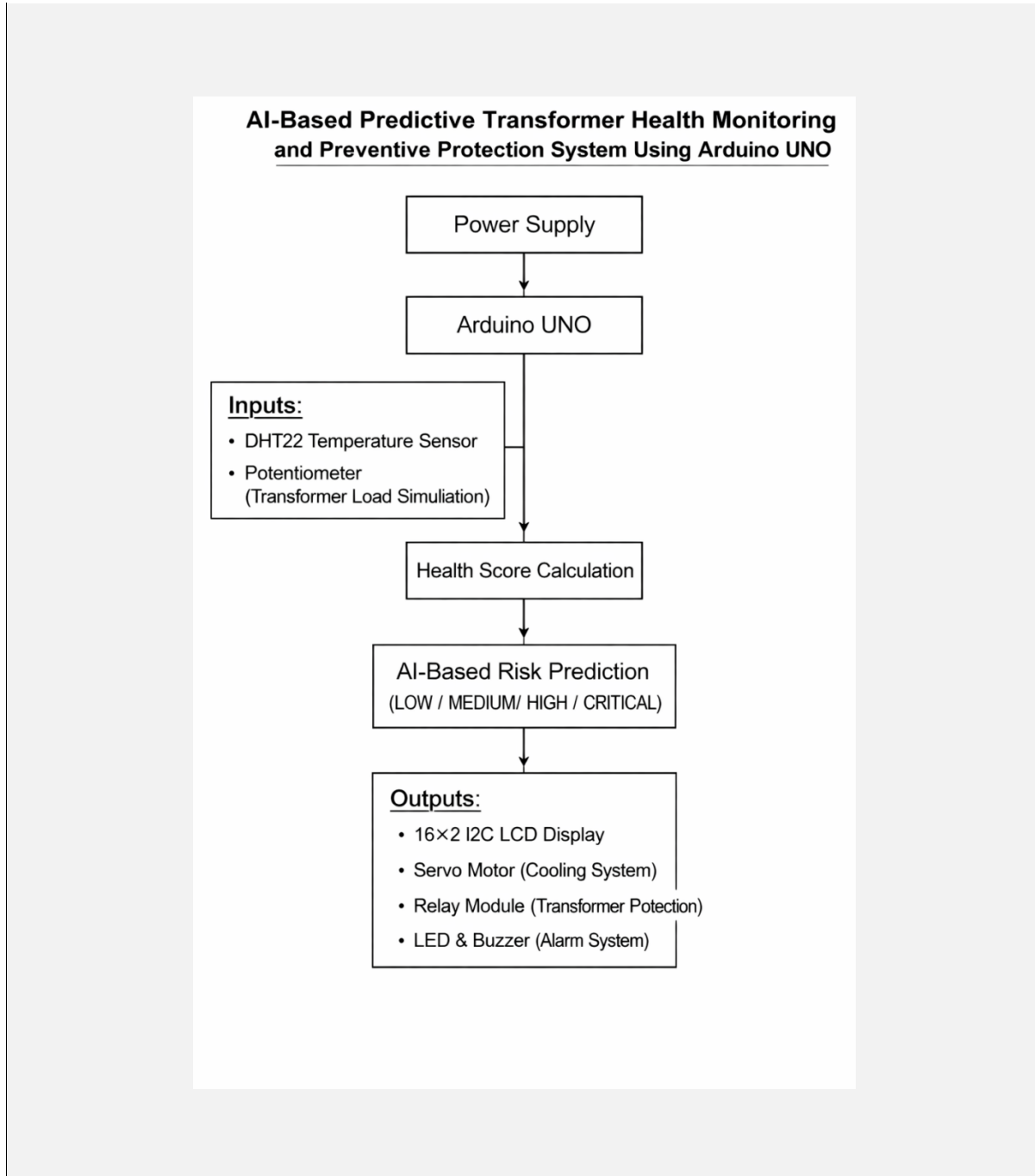


Figure 4.1 Block Diagram of the Proposed System

Power Supply → Arduino UNO → [DHT22 Sensor, Potentiometer] → Health Score and Risk Calculation → [Servo, Relay, LED, Buzzer, 16×2 LCD]

The Arduino UNO forms the central node of the block diagram. It receives inputs continuously from the DHT22 sensor and the potentiometer, computes the health score and risk classification internally, and simultaneously drives the LCD dashboard and the three protective/alert output devices, the servo, the relay, and the LED-buzzer pair.

4.5 Working Principle

The system operates on a health-score-based comparison logic rather than a single fixed threshold. Every second, the DHT22 sensor and the potentiometer are read, and the Arduino computes a health score by subtracting a temperature-based penalty, applied above 40°C, and a load-based penalty, applied above 75%, from a base value of 100.

This health score is then classified into one of four risk levels, LOW (score ≥ 80), MEDIUM (50–79), HIGH (25–49), or CRITICAL (below 25). The failure probability is derived directly as 100 minus the health score, and the remaining life is scaled proportionally from a 15-year baseline.

If the temperature crosses 45°C, the servo is rotated to a proportional cooling angle. If the risk level reaches CRITICAL, the relay disconnects the supply, the LED and buzzer switch on, and the fault counter increases by one, but only on the first cycle that the critical condition is newly detected. Every three seconds, the LCD dashboard refreshes, cycling through its four display screens.

4.6 Flowchart

Figure 4.2 presents the flowchart of the program. The oval blocks represent the start of the process and the one-time startup animation, the parallelogram blocks represent sensor reading and display update operations, the rectangular blocks represent processing steps such as health score calculation, and the diamond blocks represent the decision points at which the risk level and temperature are evaluated.

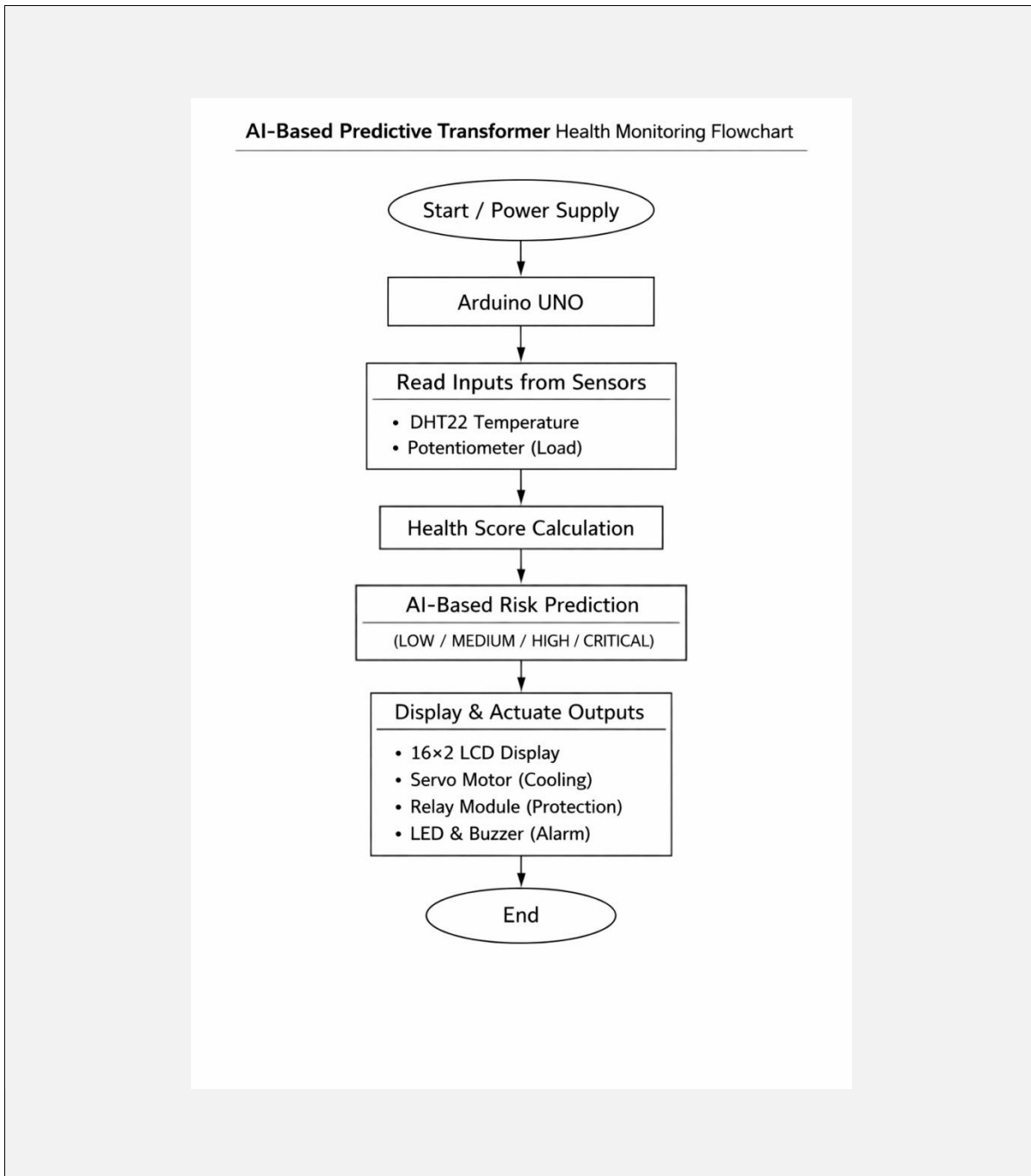


Figure 4.2 Flowchart of System Operation

As shown in the flowchart, the program first initialises the hardware and runs the startup animation, then continuously checks two independent timers. On the one-second sensor timer, it reads the sensors, calculates the health score, classifies the risk level, updates the failure probability and remaining life, checks the cooling condition, and checks the protection condition. On the separate three-second display timer, it refreshes the LCD dashboard. Both branches loop back to the same continuously repeating check, so that sensor evaluation and display refresh proceed independently without either one blocking the other.

4.7 Algorithm

The complete operation of the system follows the algorithm given below:

Step 1: Initialise the Arduino UNO, LCD, DHT22 sensor, servo, relay pin, LED pin, and buzzer pin, and set the relay to its energised, connected state.

Step 2: Run the one-time startup animation on the LCD.

Step 3: Once every second, read the temperature and the potentiometer-based load percentage.

Step 4: Calculate the health score by subtracting the temperature penalty (above 40°C) and the load penalty (above 75%) from a base of 100.

Step 5: Classify the risk level as LOW, MEDIUM, HIGH, or CRITICAL based on the health score.

Step 6: Calculate the failure probability as 100 minus the health score, and update the remaining life estimate from the 15-year baseline.

Step 7: If the temperature exceeds 45°C, rotate the servo to a proportional cooling angle; otherwise return it to its resting position.

Step 8: If the risk level is CRITICAL, de-energise the relay, switch on the LED and buzzer, and increase the fault counter once per new critical event; otherwise keep the relay energised and the LED and buzzer off.

Step 9: Once every three seconds, refresh the LCD dashboard, cycling through its four display screens.

Step 10: Repeat Steps 3 to 9 continuously for as long as the system remains powered on.

4.8 Program Description

The software performs the following tasks continuously, split across two independent timers inside the `loop()` function:

- Reads temperature from the DHT22 sensor and load from the potentiometer.
- Calculates the health score, risk level, failure probability, and remaining life.
- Activates the cooling servo when the temperature threshold is crossed.
- Activates the relay, LED, and buzzer when the risk level reaches CRITICAL.
- Refreshes the four-screen LCD dashboard.

Three external libraries were used: `LiquidCrystal_I2C` for the LCD [6], `DHT` for the temperature sensor [7], and `Servo` for the cooling fan motor [8]. The threshold values used throughout the program are summarised in Table 4.1.

Parameter	Threshold Value
Temperature Penalty Start	40°C
Load Penalty Start	75%
Cooling Activation Temperature	45°C
LOW Risk	Health Score ≥ 80
MEDIUM Risk	Health Score 50–79
HIGH Risk	Health Score 25–49
CRITICAL Risk	Health Score < 25
Baseline Remaining Life	15 years

Table 4.1 Threshold Configuration

CHAPTER 5

SIMULATION AND IMPLEMENTATION

5.1 Simulation Setup

Since this project is built and verified entirely in simulation, the implementation consists of wiring all components to the Arduino UNO inside the Wokwi simulator rather than on a physical breadboard. The DHT22 is connected to digital pin D2, the potentiometer's wiper to analog pin A0, the I2C LCD's SDA and SCL lines to A4 and A5, the servo's signal pin to D10, the relay's control pin to D11, the LED to D8, and the buzzer to D9.

The complete simulated circuit, assembled and wired inside Wokwi, is shown in Figure 5.1.

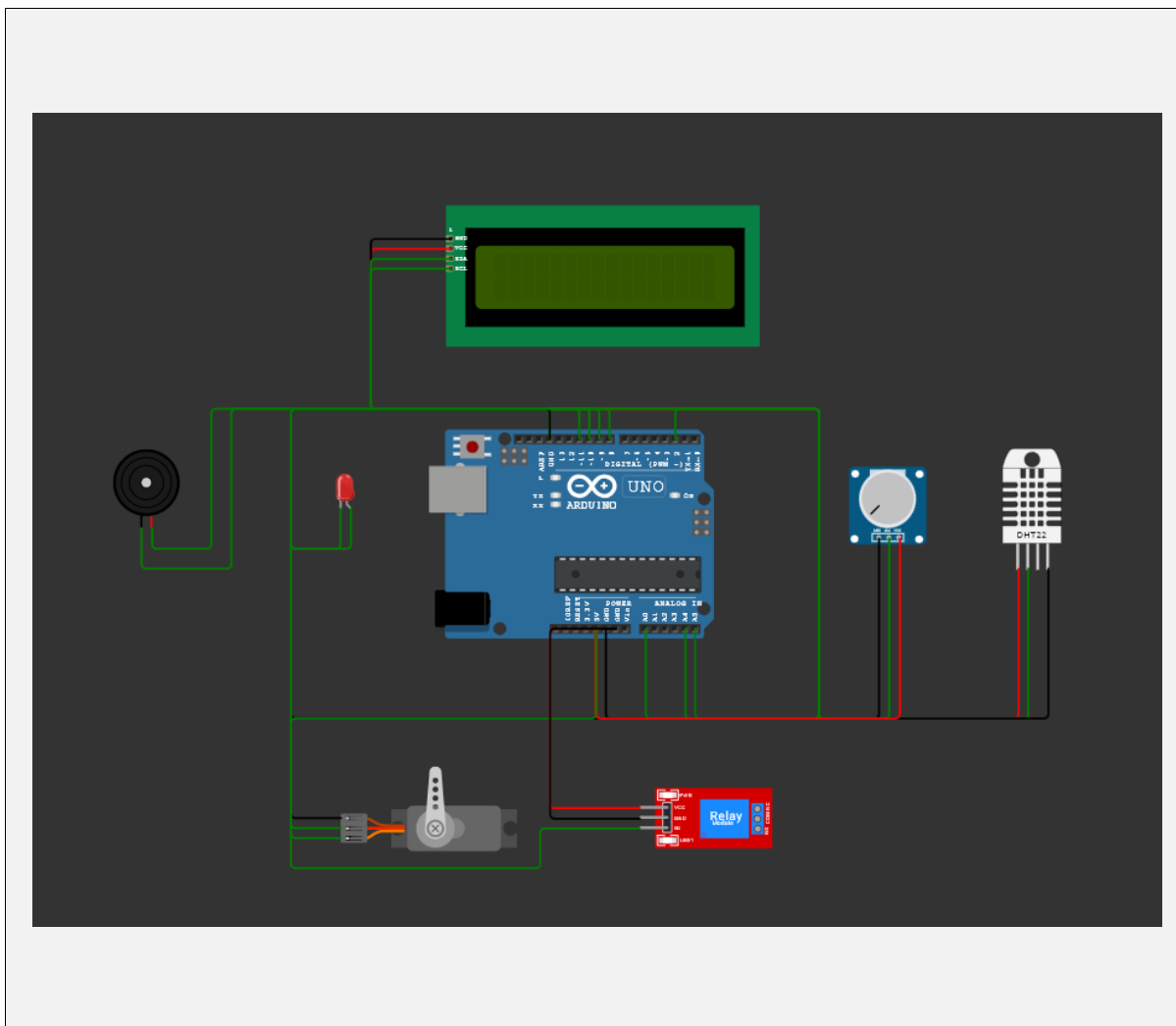


Figure 5.1 Complete Circuit Diagram – Wokwi Simulation

5.2 Circuit Diagram Explanation

The complete circuit was designed and verified inside the Wokwi simulator before being treated as final. The simulation confirmed correct pin connections, proper sensor and potentiometer interfacing, LCD dashboard rotation, servo cooling behaviour, relay tripping, and LED/buzzer alerting, all functioning together as intended.

5.3 Experimental Results

The developed system was tested inside Wokwi by varying the simulated DHT22 temperature and rotating the potentiometer to represent different load conditions. Four test cases were carried out, corresponding to the main operating states of the system.

Test Case 1 – Normal (LOW Risk) Condition

The temperature was kept below 40°C and the load below 75%. The health score remained at 100, the risk level displayed as LOW, the servo stayed at its resting position, the relay remained connected, and the LED and buzzer stayed off, as shown in Figure 5.2.

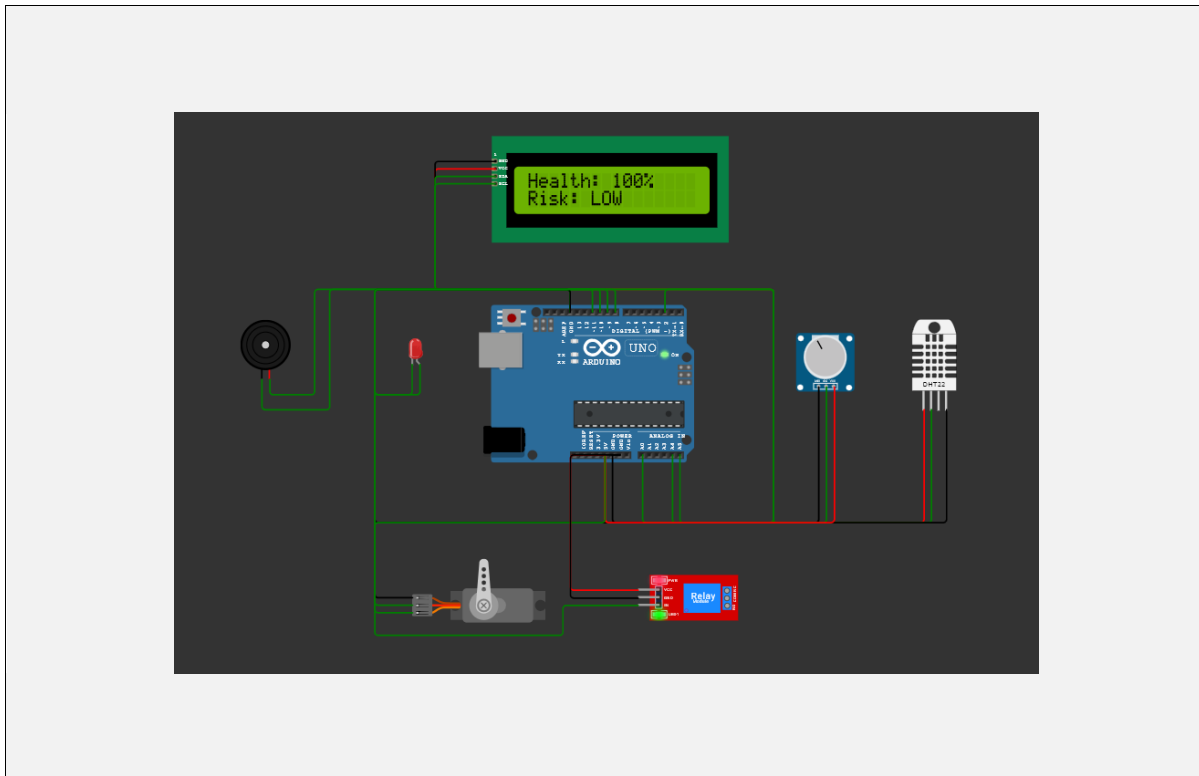


Figure 5.2 LCD Output – Normal (LOW Risk) Condition

Test Case 2 – Rising Temperature (MEDIUM/HIGH Risk)

The temperature was gradually increased past 40°C and then past 45°C, while the load was kept low. The health score dropped in line with the temperature penalty formula, the risk level moved from LOW to MEDIUM and then HIGH, and once 45°C was crossed, the servo was seen rotating proportionally further from its resting position, as shown in Figure 5.3.

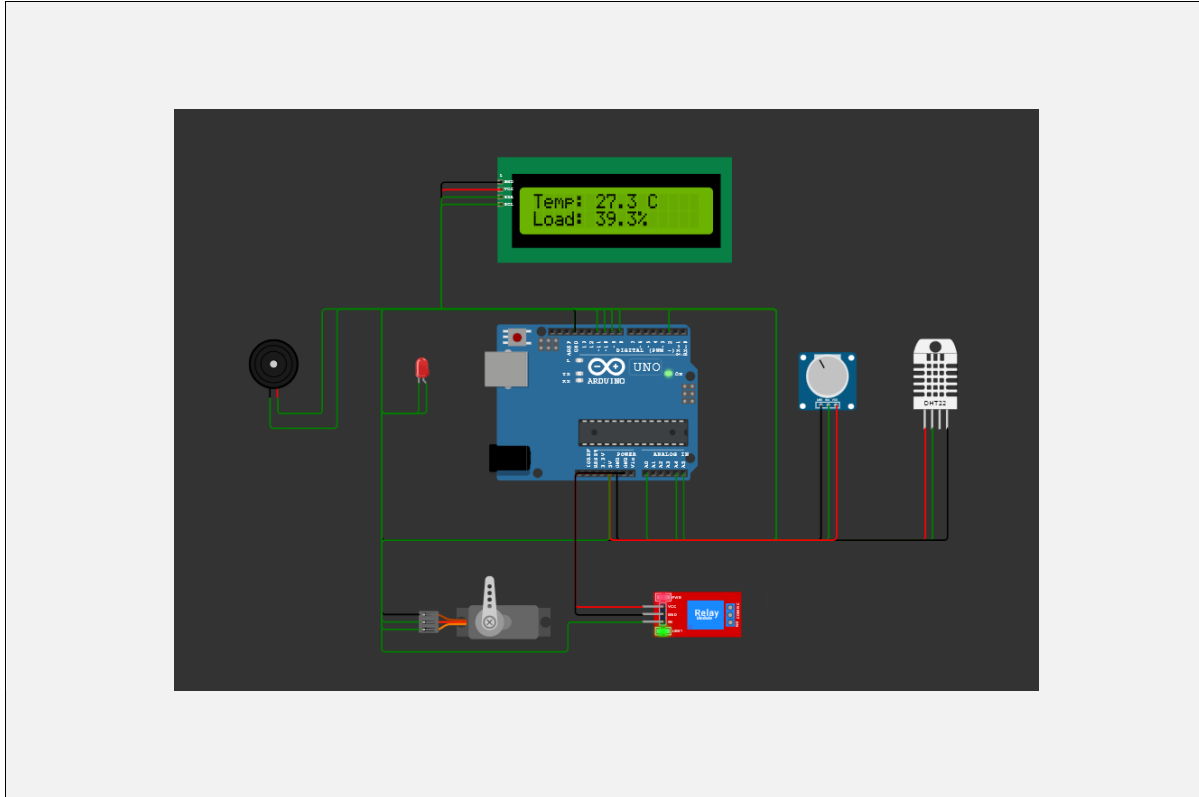


Figure 5.3 LCD Output – Rising Temperature (MEDIUM/HIGH Risk)

Test Case 3 – Overload Condition

The load was increased past 75%, up toward 100%, while the temperature was kept low. The health score dropped due to the load penalty alone, moving the risk level from LOW to MEDIUM, confirming that the load-based penalty works independently of temperature, even though load alone cannot push the score as low as the HIGH or CRITICAL range at its maximum value of 30 penalty points.

Test Case 4 – Critical Condition (Relay Trip)

Both temperature and load were increased simultaneously, pushing the health score below 25. The risk level displayed as CRITICAL, the relay was seen switching to its disconnected state, and the LED and buzzer switched on together, as shown in Figure 5.4. Repeating this test several times also confirmed that the fault counter increased by exactly one for each new critical event, without counting the same ongoing fault repeatedly.

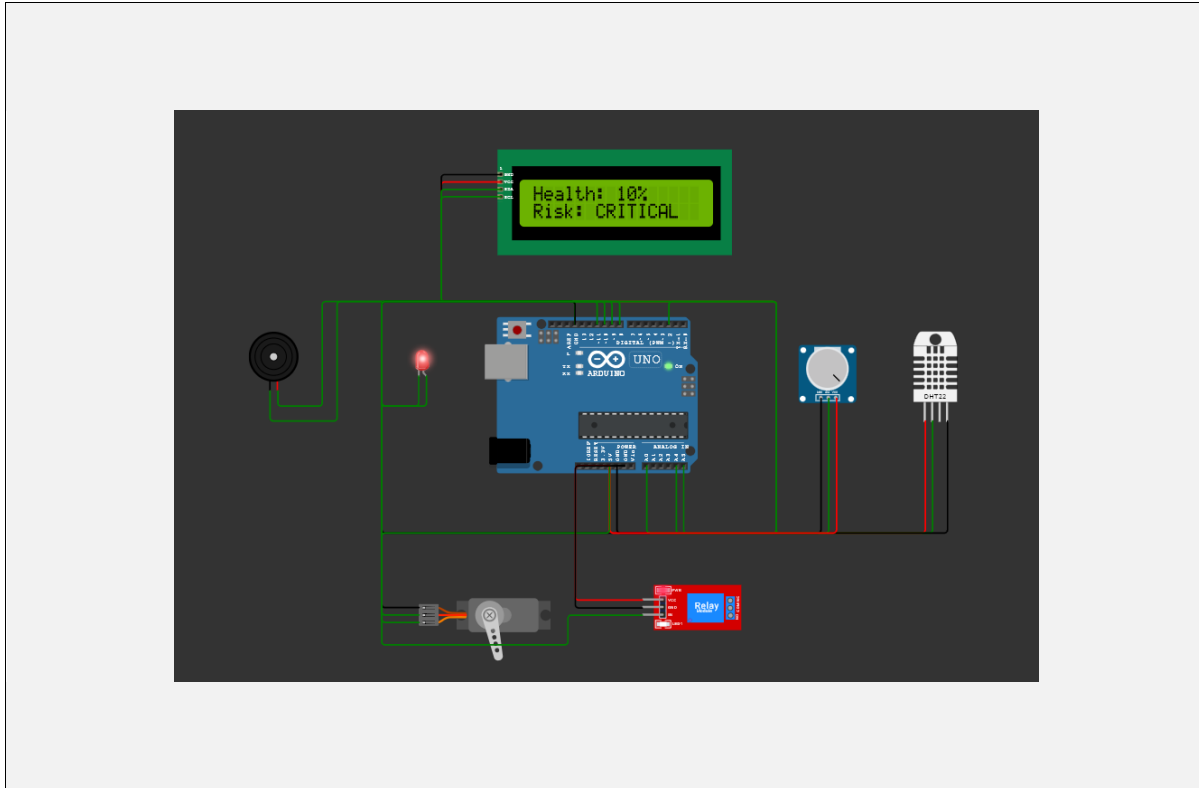


Figure 5.4 LCD Output – Critical Condition (Relay Trip)

5.4 Testing Table

Table 5.1 summarises the results obtained from all four test cases conducted during the simulation.

Test Case	Temperature	Load	Health Score	Risk Level	Relay	Result
1	32°C	40%	100	LOW	ON	Pass
2	75°C	40%	48	HIGH	ON	Pass
3	32°C	100%	70	MEDIUM	ON	Pass
4	80°C	95%	16	CRITICAL	OFF	Pass

Table 5.1 Result Summary of Test Cases

5.5 Discussion

The simulation results demonstrate that the developed system reliably distinguishes between normal, rising-risk, overload, and critical conditions using a single combined health score, rather than a fixed single-parameter threshold. The graded LOW/MEDIUM/HIGH/CRITICAL classification gave a much clearer picture of the

transformer's condition throughout each test than a simple safe-or-tripped indicator would have.

Throughout testing, the DHT22 sensor gave consistent readings, the potentiometer-based load simulation responded predictably, and the servo, relay, LED, buzzer, and LCD all reacted correctly and without noticeable delay to changing conditions. This indicates that the system is well suited for educational demonstration of predictive transformer protection concepts, and could be extended to physical hardware with minimal change to the underlying logic.

CHAPTER 6

ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

6.1 Advantages

- Combines temperature and load into a single, more meaningful health score.
- Provides four-level graded risk classification instead of a single pass-or-fail alarm.
- Gives an early, predictive warning through failure probability and remaining life estimates.
- Automatically activates proportional cooling before the condition becomes critical.
- Automatically disconnects supply through the relay during critical conditions.
- Clear visual and audible alerts through the LED and buzzer.
- Maintains a fault counter for simple maintenance record-keeping.
- Low-cost, using easily available components and a free simulation platform.
- Simple, modular code structure that is easy to test and extend.

6.2 Limitations

- Load is simulated using a potentiometer rather than an actual current or power sensor.
- The health score, risk classification, and failure probability are rule-based estimates, not derived from real historical failure data.
- The system has been tested only in simulation and not on physical hardware.
- No data logging or remote monitoring capability is currently included.
- The remaining life estimate is an indicative calculation and does not account for real ageing factors such as insulation degradation or moisture ingress.

6.3 Future Scope

- Replacing the potentiometer with an actual current or power sensor for real load measurement.
- Migrating to an ESP32 for built-in Wi-Fi connectivity and IoT-based remote monitoring.
- Logging health score and risk trends to a cloud platform for long-term analysis.
- A mobile application for real-time alerts and dashboard viewing.

- Training an actual machine learning model on real transformer failure data for more accurate failure prediction.
- Adding oil level, vibration, or humidity sensors for a more complete health picture.
- Extending the system to monitor multiple transformers from a single dashboard.

CHAPTER 7

CONCLUSION

The AI-Based Predictive Transformer Health Monitoring and Preventive Protection System was successfully designed and simulated using an Arduino UNO, a DHT22 sensor, a potentiometer, a 16×2 I2C LCD, a servo motor, a relay module, an LED, and a buzzer.

The developed system continuously monitors temperature and load, combines them into a single health score, and classifies the transformer's condition into four graded risk levels. Whenever the condition becomes dangerous, the system automatically activates cooling and, if necessary, disconnects the supply, while alerting the operator through visual, audible, and on-screen indicators.

Simulation testing confirmed that the system correctly distinguishes between normal, rising-risk, overload, and critical conditions, and that the cooling, protection, and fault-counting logic all behave as intended. This graded, predictive approach is a clear improvement over the single-threshold alarms used in many low-cost transformer protection systems.

The project demonstrates that a simple, low-cost Arduino-based system can move beyond basic ON/OFF protection and provide a meaningful, predictive view of transformer health. It satisfies the objectives set out at the beginning of this report and provides a solid foundation for future enhancement using IoT connectivity, real sensor-based load measurement, and machine learning-based failure prediction.

The developed system provides a low-cost and reliable solution for predictive transformer health monitoring and preventive protection.

REFERENCES

- [1] Arduino AG, Arduino UNO Rev3 Datasheet. Arduino AG, Somerville, MA, USA, 2021. [Online]. Available: <https://www.arduino.cc>
- [2] Aosong Electronics Co., Ltd., DHT22 Digital Temperature and Humidity Sensor Datasheet. Aosong Electronics, Guangzhou, China, 2014.
- [3] Texas Instruments, PCF8574 Remote 8-Bit I/O Expander for I2C Bus Datasheet. Texas Instruments, Dallas, TX, USA, 2015.
- [4] IEEE Std C57.91, IEEE Guide for Loading Mineral-Oil-Immersed Transformers and Step-Voltage Regulators. Institute of Electrical and Electronics Engineers, New York, NY, USA, 2011.
- [5] IEEE Std C57.104, IEEE Guide for the Interpretation of Gases Generated in Oil-Immersed Transformers. Institute of Electrical and Electronics Engineers, New York, NY, USA, 2019.
- [6] F. de Brabander, "LiquidCrystal_I2C Library," GitHub repository, 2021. [Online]. Available: https://github.com/johnrickman/LiquidCrystal_I2C
- [7] Adafruit Industries, "DHT sensor library for ESP8266 and Arduino," GitHub repository, 2023. [Online]. Available: <https://github.com/adafruit/DHT-sensor-library>
- [8] Arduino, "Servo Library Reference," Arduino Software Documentation, 2024. [Online]. Available: <https://docs.arduino.cc>
- [9] M. Banzi and M. Shiloh, Getting Started with Arduino, 3rd ed. Sebastopol, CA, USA: Maker Media, 2015.
- [10] Wokwi, "Wokwi Arduino and ESP32 Simulator Documentation," Wokwi Online Simulator Documentation, 2024. [Online]. Available: <https://docs.wokwi.com>

APPENDIX A

ARDUINO SOURCE CODE

This appendix contains the complete Arduino source code used in this project, exactly as simulated on Wokwi, included here for reference against the Algorithm and Program Description sections of this report.

```
/*
-----
Project Name : AI-Based Predictive Transformer Health
               Monitoring and Preventive Protection System
Board       : Arduino UNO
Sensors     : DHT22, Potentiometer (Load Simulation)
Display     : 16x2 I2C LCD
Alert System : LED, Buzzer, Relay Module, Servo Motor
Author      : Karthikeyan M
Year       : 2026
-----
*/
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>
#include <Servo.h>

// --- Pin Definitions ---
#define DHTPIN 2
#define DHTTYPE DHT22
#define LED_PIN 8
#define BUZZER_PIN 9
#define SERVO_PIN 10
#define RELAY_PIN 11
#define POT_PIN A0

// --- Constants & Thresholds ---
const unsigned long SCREEN_INTERVAL = 3000; // 3 seconds per LCD dashboard screen
const unsigned long SENSOR_INTERVAL = 1000; // Read sensors every 1 second

// --- Global Objects ---
LiquidCrystal_I2C lcd(0x27, 16, 2);
DHT dht(DHTPIN, DHTTYPE);
Servo coolingServo;

// --- System State Variables ---
float temperature = 0.0;
float currentLoad = 0.0;
int healthScore = 100;
String riskLevel = "LOW";
float remainingLifeYears = 15.0;
int failureProbability = 0;
int faultCount = 0;
```

```

bool relayState = true;           // true = ON (Healthy), false = OFF
                                   (Tripped/Critical)
bool wasCriticalLastCheck = false; // Flag to prevent continuous increment of fault
                                   counter

// --- Timing Variables ---
unsigned long lastScreenUpdate = 0;
unsigned long lastSensorRead = 0;
int currentScreen = 0;

// --- Function Prototypes ---
void startupAnimation();
void readSensors();
void calculateHealth();
void predictRisk();
void estimateRemainingLife();
void calculateFailureProbability();
void controlCooling();
void controlProtection();
void displayDashboard();

void setup() {
    // Initialize Serial Communication for debugging
    Serial.begin(9600);

    // Initialize Peripherals & Pins
    pinMode(LED_PIN, OUTPUT);
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(RELAY_PIN, OUTPUT);

    // Set initial safe states (relay module used here is active-HIGH: HIGH =
    energised/connected)
    digitalWrite(RELAY_PIN, HIGH); // Relay ON initially (Energized / Connected)
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);

    coolingServo.attach(SERVO_PIN);
    coolingServo.write(0); // Fan/Valve shut off position

    dht.begin();

    lcd.init();
    lcd.backlight();

    // Execute Mandatory Startup Sequence
    startupAnimation();
}

void loop() {
    unsigned long currentMillis = millis();

    // Periodic Sensor Evaluation and AI Modeling Execution
    if (currentMillis - lastSensorRead >= SENSOR_INTERVAL) {
        lastSensorRead = currentMillis;
    }
}

```

```

    readSensors();
    calculateHealth();
    predictRisk();
    estimateRemainingLife();
    calculateFailureProbability();
    controlCooling();
    controlProtection();
}

// Periodic LCD Dashboard Rotation Screen Manager
if (currentMillis - lastScreenUpdate >= SCREEN_INTERVAL) {
    lastScreenUpdate = currentMillis;
    displayDashboard();
    currentScreen = (currentScreen + 1) % 4; // Cycles through screens 0, 1, 2, 3
}
}

// 1. Professional Startup Animation
void startupAnimation() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("SMART TRANSFORMER");
    delay(1500);

    lcd.setCursor(0, 1);
    lcd.print("AI ENGINE STARTING");
    for (int i = 0; i < 3; i++) {
        delay(500);
    }

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("SYSTEM READY");
    delay(1500);
    lcd.clear();
}

// 2. Read Sensors (Temperature and Load)
void readSensors() {
    float tempRead = dht.readTemperature();
    // Ensure the reading is valid before overwriting global state
    if (!isnan(tempRead)) {
        temperature = tempRead;
    }

    // Read Potentiometer and map to percentage load (0 to 100%)
    int potValue = analogRead(POT_PIN);
    currentLoad = (potValue / 1023.0) * 100.0;
}

// 3. Calculate Health Score (0 - 100%)
void calculateHealth() {
    // Analytical model combining temperature and load penalties

```

```

float tempPenalty = 0.0;
float loadPenalty = 0.0;

// Temperature Penalty: Safe up to 40°C, linear degradation up to 85°C
if (temperature > 40.0) {
    tempPenalty = (temperature - 40.0) * 1.5;
}

// Load Penalty: Rated load up to 75%, linear degradation beyond nominal limit
if (currentLoad > 75.0) {
    loadPenalty = (currentLoad - 75.0) * 1.2;
}

int computedHealth = 100 - (int)(tempPenalty + loadPenalty);

// Constrain bounds between 0% and 100%
if (computedHealth > 100) computedHealth = 100;
if (computedHealth < 0) computedHealth = 0;

healthScore = computedHealth;
}

// 4. Predict Risk Level (LOW, MEDIUM, HIGH, CRITICAL)
void predictRisk() {
    if (healthScore >= 80) {
        riskLevel = "LOW";
    } else if (healthScore >= 50 && healthScore < 80) {
        riskLevel = "MEDIUM";
    } else if (healthScore >= 25 && healthScore < 50) {
        riskLevel = "HIGH";
    } else {
        riskLevel = "CRITICAL";
    }
}

// 5. Estimate Remaining Life (Years)
void estimateRemainingLife() {
    // Baseline transformer operational life span = 15 years
    // Remaining life correlates to current structural/thermal health
    remainingLifeYears = (healthScore / 100.0) * 15.0;
    if (remainingLifeYears < 0.0) {
        remainingLifeYears = 0.0;
    }
}

// 6. Failure Probability Calculations
void calculateFailureProbability() {
    // Failure probability is inversely proportional to health score
    failureProbability = 100 - healthScore;
}

// 7. Servo Automatic Cooling Actuation
void controlCooling() {
    // Dynamic positioning based on temperature profile

```

```

    if (temperature > 45.0) {
        // Open cooling valve / turn fan completely (90 degrees to 180 degrees
        proportional sweep)
        float sweep = map(temperature, 45, 85, 90, 180);
        int angle = constrain((int)sweep, 90, 180);
        coolingServo.write(angle);
    } else {
        // Retract servo to home positioning if normal thermal thresholds met
        coolingServo.write(0);
    }
}

// 8. Relay Protection, LED, and Buzzer Systems
void controlProtection() {
    if (riskLevel == "CRITICAL") {
        relayState = false; // Tripped state
        digitalWrite(RELAY_PIN, LOW); // De-energize system / Safe state isolation

        // Actuate visual and audible indicators
        digitalWrite(LED_PIN, HIGH);
        digitalWrite(BUZZER_PIN, HIGH);

        // Edge-triggered fault counter incrementation
        if (!wasCriticalLastCheck) {
            faultCount++;
            wasCriticalLastCheck = true;
        }
    } else {
        relayState = true;
        digitalWrite(RELAY_PIN, HIGH); // Re-engage grid contactor

        digitalWrite(LED_PIN, LOW);
        digitalWrite(BUZZER_PIN, LOW);

        wasCriticalLastCheck = false; // Reset trigger condition when cleared
    }
}

// 12. LCD Dashboard Engine (Exactly 4 Screens)
void displayDashboard() {
    lcd.clear();
    switch(currentScreen) {
        case 0:
            // Screen 1: Health & Risk Assessment
            lcd.setCursor(0, 0);
            lcd.print("Health: ");
            lcd.print(healthScore);
            lcd.print("%");
            lcd.setCursor(0, 1);
            lcd.print("Risk: ");
            lcd.print(riskLevel);
            break;

        case 1:

```

```

        // Screen 2: Real-time Telemetry parameters
        lcd.setCursor(0, 0);
        lcd.print("Temp: ");
        lcd.print(temperature, 1);
        lcd.print(" C");
        lcd.setCursor(0, 1);
        lcd.print("Load: ");
        lcd.print(currentLoad, 1);
        lcd.print("%");
        break;

    case 2:
        // Screen 3: Algorithmic Predictions
        lcd.setCursor(0, 0);
        lcd.print("Rem.Life: ");
        lcd.print(remainingLifeYears, 1);
        lcd.print("Yrs");
        lcd.setCursor(0, 1);
        lcd.print("Failure: ");
        lcd.print(failureProbability);
        lcd.print("%");
        break;

    case 3:
        // Screen 4: Operational Logbook
        lcd.setCursor(0, 0);
        lcd.print("Fault Count: ");
        lcd.print(faultCount);
        lcd.setCursor(0, 1);
        lcd.print("Relay: ");
        lcd.print(relayState ? "ON" : "OFF");
        break;
    }
}

```


APPENDIX B

WOKWI SIMULATION LINK

Project Name:

AI-Based Predictive Transformer Health Monitoring and Preventive Protection System
Using Arduino UNO

Simulation Platform:

Wokwi Online Simulator

Simulation Link:

<https://wokwi.com/projects/469066635947975681>

The complete project circuit and Arduino program were designed, tested, and verified using the Wokwi Online Simulator. The simulation validates the working of the proposed predictive transformer health monitoring and preventive protection system under different operating conditions.